Resumo do projeto

Book Store API com Cypress — o que foi feito e como executar ou dar manutenção

O projeto testa a API Book Store da DemoQA (`https://bookstore.toolsqa.com`) com Cypress. A suíte cobre listagem de livros, criação de usuário, geração de token e autorização. A reserva de livros ainda não foi feita.

O que ficou pronto

A listagem de livros em `cypress/e2e/books/listar-livros.cy.js` já existia e não foi alterada. Ela valida o `GET /BookStore/v1/Books`: status 200, array `books` e os campos de cada livro.

A conta foi separada em três specs, cada um executável sozinho:

- `cypress/e2e/account/criar-usuario.cy.js` cria um usuário com nome exclusivo e valida o status 201, o `userID` e o `username`.
- `cypress/e2e/account/gerar-token.cy.js` cria o usuário e valida o token com status 200 e `Success`.
- `cypress/e2e/account/validar-autorizacao.cy.js` cria o usuário, gera o token e só então autoriza. Nesta API, `POST /Account/v1/Authorized` responde `false` até o token existir.

As chamadas ficam em `cypress/support/api/account-api.js`: `criarUsuario()`, `gerarToken()` e `validarAutorizacao()`. A senha e o prefixo do usuário vêm do `.env`. Senha e token não aparecem no log.

O `cypress.config.js` guarda a URL base, carrega o `.env` com dotenv e gera o relatório HTML. A última execução passou: 4 testes, 4 ok.

Como executar

Na primeira vez, nesta pasta:

```
npm install
cp .env.example .env
```

No `.env`, preencha `BOOKSTORE_PASSWORD` com uma senha aceita pela API: 8 caracteres ou mais, com maiúscula, minúscula, número e caractere especial. O `BOOKSTORE_USERNAME_PREFIX` pode ficar `usuario`. O `.env` não entra no Git.

Objetivo	Comando
Rodar tudo no terminal	`npm test`
Ver um teste no Chrome	`npx cypress run --headed --no-exit --browser chrome --spec "caminho/do/spec"`

Objetivo	Comando
Escolher o teste na interface	`npm run cy:open`
Gerar o relatório HTML	`npm run test:report`

O relatório abre em `cypress/results/relatorio-bookstore.html`. Para ver os testes na ordem do fluxo, use estes specs, um por vez, fechando o Chrome antes do próximo:

```
npx cypress run --headed --no-exit --browser chrome --spec
"cypress/e2e/account/criar-usuario.cy.js"
npx cypress run --headed --no-exit --browser chrome --spec
"cypress/e2e/account/gerar-token.cy.js"
npx cypress run --headed --no-exit --browser chrome --spec
"cypress/e2e/account/validar-autorizacao.cy.js"
npx cypress run --headed --no-exit --browser chrome --spec
"cypress/e2e/books/listar-livros.cy.js"
```

Para ver os quatro seguidos na mesma janela:

```
npx cypress run --headed --no-exit --browser chrome
```

Como dar manutenção

- Novo teste de conta: reutilize as funções de `cypress/support/api/account-api.js`. Não copie a URL completa nem a senha para dentro do spec.
- Novo endpoint: crie a função no módulo de API e o spec em `cypress/e2e/`, com asserções no teste e a requisição na função.
- Cada spec de conta prepara o próprio usuário no `before()`, porque o Cypress isola os arquivos. Não dependa de um usuário criado por outro spec.
- A URL base muda só em `cypress.config.js`.
- Senha e prefixo mudam só no `.env`.
- O próximo passo planejado é reservar dois livros e consultar o usuário.

> Arquivo gerado como resumo de trabalho do projeto **desafio-bookstore-api-cypress**. Não contém senha nem token.